

INSTITUTO FEDERAL DO PARANÁ
JOÃO PEDRO DOS SANTOS HENRIQUE PLINTA

ÁRVORE B COM COMPRESSÃO DE DADOS

SUMÁRIO

Sumário	1
1 INTRODUÇÃO	2
2 A ÁRVORE B	2
2.1 Definição e propriedades	2
2.2 Implementação	3
2.2.1 Inserção	3
2.2.2 Busca	3
2.2.3 Remoção	3
2.3 Persistência em arquivo	4
2.4 Dois graus escolhidos	4
2.5 Métricas coletadas	4
3 ALGORITMOS DE COMPRESSÃO	5
3.1 LZW (<i>Lempel–Ziv–Welch</i>)	5
3.1.1 Compressão	5
3.1.2 Descompressão	5
3.1.3 Formato do arquivo	6
3.2 Huffman Canônico	6
3.2.1 Compressão	6
3.2.2 Descompressão	6
3.2.3 Formato do arquivo	7
4 TESTES E RESULTADOS	7
4.1 Ambiente de testes	7
4.2 Desempenho de inserção	7
4.3 Desempenho de busca	8
4.4 Desempenho de remoção	8
4.5 Memória utilizada	9
4.6 Compressão	9
5 ANÁLISE E DISCUSSÃO	9
5.1 Impacto do grau T no desempenho	9
5.2 Comparação entre LZW e Huffman	10
5.3 Integridade dos dados	11
6 CONCLUSÃO	11

1 INTRODUÇÃO

Este relatório descreve a implementação e os resultados do Trabalho 2 da disciplina de Estruturas de Dados Avançadas. O trabalho consiste em:

1. Implementar uma **Árvore B** com chaves do tipo *string*, com suporte a inserção, busca, remoção e persistência em arquivo binário;
2. Implementar dois **algoritmos de compressão** — LZW e Huffman Canônico — para comprimir o arquivo gerado pela árvore;
3. Medir e comparar o desempenho de ambas as estruturas em diferentes cenários.

A motivação é simular o ambiente real de sistemas operacionais e bancos de dados, que utilizam árvores B para indexação eficiente e precisam persistir essas estruturas em dispositivos de armazenamento. A compressão reduz o espaço necessário e o tempo de leitura/escrita em disco.

O trabalho é composto por dois programas independentes escritos em **C++17**:

Programa	Função
btree	Gerencia a árvore B interativamente via linha de comando
compress	Comprime e descomprime o arquivo da árvore

2 A ÁRVORE B

2.1 Definição e propriedades

A Árvore B com grau mínimo T satisfaz as seguintes propriedades:

- Todo nó tem no máximo $2T - 1$ chaves;
- Todo nó interno (exceto a raiz) tem no mínimo $T - 1$ chaves;
- A raiz tem no mínimo 1 chave (se a árvore não for vazia);
- Todo nó interno com k chaves tem exatamente $k + 1$ filhos;
- Todas as folhas estão no mesmo nível;
- As chaves em cada nó estão em ordem crescente.

A altura de uma Árvore B com N chaves é $O(\log_T N)$, o que garante custo logarítmico para inserção, busca e remoção.

2.2 Implementação

A implementação utiliza as seguintes estruturas em C++:

```
1 struct Node {
2     int n;                      // numero de chaves
3     bool leaf;
4     std::vector<std::string> keys; // chaves[0..n-1]
5     std::vector<Node*> ch;        // filhos[0..n]
6     Node(int t, bool lf) : n(0), leaf(lf),
7         keys(2*t-1), ch(2*t, nullptr) {}
8 };
```

As chaves são armazenadas como `std::string`, com comparação lexicográfica. O grau mínimo T é definido na linha de comando.

2.2.1 Inserção

A inserção utiliza a estratégia *proativa*: ao descer pela árvore, nós cheios ($n = 2T - 1$) são divididos (*split*) antes de continuar a descida. Isso evita que seja necessário subir novamente após a inserção na folha. A complexidade é $O(T \cdot \log_T N)$.

2.2.2 Busca

A busca compara a chave com as chaves do nó atual da esquerda para a direita. Ao encontrar a posição correta, desce para o filho correspondente ou retorna “não encontrado” em uma folha. Complexidade: $O(T \cdot \log_T N)$.

2.2.3 Remoção

A remoção trata três casos:

1. **Chave em folha**: remoção direta com deslocamento dos vizinhos;
2. **Chave em nó interno com filho suficientemente cheio**: substitui pela chave predecessora ou sucessora e remove recursivamente;
3. **Chave em nó interno com filhos “magros”**: realiza fusão (*merge*) dos filhos e desce recursivamente.

2.3 Persistência em arquivo

O arquivo binário é escrito em **pré-ordem** (raiz, filho₀, filho₁, ...). Cada nó é serializado como:

Campo	Tipo	Descrição
leaf	int (4 B)	1 se folha, 0 se interno
n	int (4 B)	número de chaves
len _{<i>i</i>}	int (4 B)	comprimento da <i>i</i> -ésima chave
key _{<i>i</i>}	char[len _{<i>i</i>}]	bytes da <i>i</i> -ésima chave

O cabeçalho do arquivo armazena o grau T , permitindo carregar a árvore corretamente mesmo que o programa seja iniciado com um grau diferente. O uso de tamanho variável por chave evita o desperdício de bytes nulos que ocorreria com campos de tamanho fixo.

2.4 Dois graus escolhidos

Foram escolhidos dois graus bastante diferentes para observar o comportamento em dois extremos:

Propriedade	$T = 3$	$T = 50$
Mín. de chaves por nó	2	49
Máx. de chaves por nó	5	99
Mín. de filhos por nó interno	3	50
Máx. de filhos por nó interno	6	100
Fator de ramificação	baixo	alto
Altura esperada (100k chaves)	≈ 11	≈ 3

$T = 3$ representa uma árvore com nós pequenos e muitos níveis, similar a uma Árvore Binária de Busca balanceada (porém com fator de ramificação 3). $T = 50$ é o perfil típico de bancos de dados e sistemas de arquivos, onde cada nó corresponde a um bloco de disco e convém armazenar o máximo de chaves por acesso.

2.5 Métricas coletadas

Após cada operação de inserção, busca ou remoção, o programa exibe:

```
[nos visitados: 6 | tempo: 1.28e-06 s | memoria: 4480 KB]
```

Nós visitados Incrementado no início de cada chamada recursiva que acessa um nó.

Tempo Medido com `std::chrono::steady_clock` antes e depois da operação.

Memória Lida de `VmRSS` em `/proc/self/status` após a operação (memória física residente do processo).

3 ALGORITMOS DE COMPRESSÃO

3.1 LZW (*Lempel–Ziv–Welch*)

O LZW é um algoritmo de compressão baseado em dicionário sem parâmetros de configuração. O dicionário é inicializado com as 256 entradas correspondentes a cada byte possível e cresce dinamicamente ao longo da compressão.

3.1.1 Compressão

O algoritmo mantém uma *string* corrente w inicialmente vazia. Para cada byte c lido:

- Se $w + c$ está no dicionário, $w \leftarrow w + c$;
- Caso contrário, emite o código de w , adiciona $w + c$ ao dicionário e $w \leftarrow c$.

Ao final, emite o código de w .

A implementação usa códigos de **16 bits** (`uint16_t`), permitindo até 65.536 entradas. Quando o dicionário está cheio, ele é congelado (sem *reset*).

3.1.2 Descompressão

O descompressor mantém o mesmo dicionário (reconstruído localmente) e inverte o processo. O único caso especial ocorre quando o código recebido ainda não foi adicionado ao dicionário — nesse caso, a entrada é $w + w[0]$ (onde w é a última entrada processada).

3.1.3 Formato do arquivo

Campo	Tamanho	Conteúdo
Magic	4 B	LZW\0
Tamanho original	8 B	uint64_t LE
Número de códigos	8 B	uint64_t LE
Códigos	$N \times 2$ B	uint16_t LE cada

3.2 Huffman Canônico

O algoritmo de Huffman atribui códigos de comprimento variável a cada símbolo (byte), com códigos mais curtos para símbolos mais frequentes. A variante **canônica** garante que, dados apenas os comprimentos dos códigos, os códigos podem ser reconstruídos deterministicamente — eliminando a necessidade de serializar a árvore inteira no arquivo.

3.2.1 Compressão

1. Contar a frequência de cada byte no arquivo de entrada;
2. Construir a árvore de Huffman com uma fila de prioridade mínima;
3. Coletar os comprimentos dos códigos para cada um dos 256 símbolos possíveis;
4. Gerar os códigos canônicos: ordenar símbolos por (comprimento, valor) e atribuir códigos inteiros em ordem crescente, deslocando à esquerda a cada mudança de comprimento;
5. Codificar os dados bit a bit (MSB primeiro dentro de cada byte).

3.2.2 Descompressão

O descompressor lê os 256 comprimentos do cabeçalho, reconstrói os mesmos códigos canônicos e decodifica o fluxo de bits símbolo a símbolo até que `tamanho_original` bytes sejam produzidos.

3.2.3 Formato do arquivo

Campo	Tamanho	Conteúdo
Magic	4 B	HUF\0
Tamanho original	8 B	uint64_t LE
Padding	1 B	bits de preenchimento no último byte
Comprimentos	256 B	1 byte por símbolo (0 = ausente)
Dados	variável	fluxo de bits codificados

O cabeçalho tem tamanho fixo de **269 bytes**, o que explica a menor eficiência em arquivos pequenos.

4 TESTES E RESULTADOS

4.1 Ambiente de testes

Componente	Especificação
Sistema operacional	Linux 6.6 (WSL2)
Compilador	g++ 13, flags -O2 -std=c++17
Geração de chaves	<i>strings</i> aleatórias de 4–10 letras minúsculas
Semente	42 (reprodutível)

4.2 Desempenho de inserção

Para cada combinação de N (número de chaves) e T (grau mínimo), foram inseridas N chaves e calculadas as médias por operação.

Tabela 1 – Desempenho médio de inserção

N	$T = 3$		$T = 50$		Redução (T50 vs T3)	
	Nós visit.	Tempo (μs)	Nós visit.	Tempo (μs)	Nós	Tempo
1.000	4,55	0,92	1,90	1,08	−58%	−17%
10.000	6,35	1,28	2,29	1,44	−64%	−12%
100.000	8,08	1,80	2,93	2,02	−64%	−12%

O crescimento dos nós visitados segue a razão logarítmica esperada. Para $T = 3$, o número de nós visitados cresce de 4,55 para 8,08 ao aumentar N de 1.000 para 100.000 (aumento

de $100\times$ em N , aumento de $\approx 1,8\times$ em nós visitados). Para $T = 50$, o crescimento é ainda menor: de 1,90 para 2,93 ($\approx 1,5\times$).

Embora $T = 50$ visite menos nós, seu tempo por operação é *ligeiramente* maior. Isso se deve ao fato de que cada nó com $T = 50$ tem mais chaves para comparar internamente — o custo de varredura linear dentro do nó é maior, parcialmente compensando o ganho em profundidade.

4.3 Desempenho de busca

Foram realizadas 400 buscas após inserir 100.000 chaves: 200 em chaves existentes e 200 em chaves inexistentes.

Tabela 2 – Desempenho de busca com $N = 100.000$ chaves

Operação	$T = 3$ (nós visit.)	$T = 50$ (nós visit.)
Busca (existente + inexistente)	7,89	2,96

O comportamento é consistente com a inserção: $T = 50$ visita aproximadamente $2,7\times$ menos nós. As buscas por chaves inexistentes percorrem a árvore até uma folha, visitando tantos nós quanto a altura da árvore.

4.4 Desempenho de remoção

Foram removidas 10% das chaves (10.000) após inserir 100.000.

Tabela 3 – Desempenho de remoção com $N = 100.000$ chaves

	$T = 3$	$T = 50$
Média de nós visitados	8,01	3,00

A remoção visita ligeiramente mais nós do que a busca, pois pode precisar realizar *merge* ou empréstimo de chaves entre nós irmãos, o que envolve acessos adicionais.

4.5 Memória utilizada

Tabela 4 – Memória residente ao final da inserção de N chaves

N	$T = 3$ (KB)	$T = 50$ (KB)
1.000	3.712	3.712
10.000	4.480	4.096
100.000	12.800	9.088

Com $T = 50$, a memória é cerca de **29% menor** para 100.000 chaves. Isso ocorre pois há menos nós no total (a árvore é mais larga e rasa), apesar de cada nó individual ser maior.

4.6 Compressão

As árvores foram salvas com $T = 3$ e comprimidas com ambos os algoritmos.

Tabela 5 – Resultados de compressão dos arquivos de árvore

N	Original	LZW		Huffman	
		Comprimido	Taxa	Comprimido	Taxa
1.000	13,2 KB	9,9 KB	1,34×	7,3 KB	1,81×
10.000	132,1 KB	73,4 KB	1,80×	70,4 KB	1,88×
100.000	1.318,5 KB	675,0 KB	1,95×	700,4 KB	1,88×

Tabela 6 – Tempo e memória de compressão

N	LZW		Huffman	
	Tempo	Memória	Tempo	Memória
1.000	2,0 ms	3.772 KB	0,7 ms	3.328 KB
10.000	13,1 ms	5.976 KB	3,4 ms	3.584 KB
100.000	120,0 ms	9.496 KB	25,4 ms	5.376 KB

5 ANÁLISE E DISCUSSÃO

5.1 Impacto do grau T no desempenho

O resultado mais significativo é a diferença no número de nós visitados entre $T = 3$ e $T = 50$. Com 100.000 chaves, $T = 3$ visita em média **8,08 nós** por inserção, enquanto $T = 50$

visita apenas **2,93**. Isso reflete diretamente a diferença de altura:

$$h_{T=3}(N) = \left\lceil \log_3 \frac{N+1}{2} \right\rceil \quad h_{T=50}(N) = \left\lceil \log_{50} \frac{N+1}{2} \right\rceil$$

Para $N = 100.000$: $h_{T=3} \approx 11$ e $h_{T=50} \approx 3$.

Em sistemas de banco de dados reais, onde cada nó representa um bloco de disco (tipicamente 4–16 KB), a redução de ≈ 8 para ≈ 3 acessos a disco por operação representa uma melhoria expressiva. Com latência de disco de ≈ 5 ms por acesso, a economia seria de ≈ 25 ms por operação.

No contexto em memória principal (como neste trabalho), a vantagem de T grande é menor porque o custo de acesso a nós distintos é baixo. De fato, o tempo por operação com $T = 50$ é apenas $\approx 12\%$ menor do que com $T = 3$, pois o custo de varredura linear dentro de nós maiores parcialmente compensa a menor profundidade.

5.2 Comparação entre LZW e Huffman

O comportamento dos dois algoritmos varia conforme o tamanho do arquivo:

- Para $N = 1.000$ e $N = 10.000$: Huffman supera o LZW tanto em taxa ($1,81\times/1,88\times$ vs $1,34\times/1,80\times$) quanto em tempo ($\approx 3\text{--}5\times$ mais rápido);
- Para $N = 100.000$: LZW supera o Huffman em taxa ($1,95\times$ vs $1,88\times$), mas ainda é $\approx 5\times$ mais lento;
- **Memória:** a memória do LZW cresce com o arquivo ($3,8\text{ MB} \rightarrow 9,5\text{ MB}$), pois o dicionário de *strings* cresce até 65.536 entradas; o Huffman mantém uso estável ($\approx 3,3\text{--}5,4\text{ MB}$) independentemente do tamanho.

O LZW melhora expressivamente com arquivos maiores (de $1,34\times$ para $1,95\times$) porque seu dicionário acumula padrões de *seqüências* de bytes à medida que processa o arquivo. O arquivo da Árvore B é rico nesses padrões: cabeçalhos de nó (leaf e n) se repetem frequentemente, os comprimentos das chaves formam seqüências curtas previsíveis, e o conteúdo alfabético das chaves cria bigramas e trigramas recorrentes. Com 100.000 chaves, o dicionário de 65.536 entradas do LZW fica totalmente preenchido e bem sintonizado para o arquivo.

O Huffman é mais estável (de $1,81\times$ para $1,88\times$) porque explora apenas a frequência individual de cada byte, que converge rapidamente. Para arquivos pequenos, onde o LZW ainda não acumulou padrões suficientes no dicionário, o Huffman leva vantagem clara. Para arquivos grandes, o LZW inverte o resultado.

5.3 Integridade dos dados

Em todos os testes, os arquivos descomprimidos foram verificados com cmp e são **bit a bit idênticos** aos originais. Isso confirma a corretude dos dois algoritmos de compressão e descompressão.

6 CONCLUSÃO

O trabalho demonstrou que a *Árvore B* é uma estrutura de indexação eficiente mesmo para centenas de milhares de chaves, com operações na casa dos microssegundos e crescimento logarítmico no número de nós acessados.

A escolha do grau T tem impacto direto no desempenho: graus maiores reduzem a altura da árvore e o número de nós visitados, ao custo de nós maiores individualmente. Para sistemas orientados a disco, onde o acesso por nó domina o custo total, T grande é claramente superior. Em memória principal, o ganho é menor mas ainda presente (tanto em nós visitados quanto em memória total).

Em relação à compressão, os resultados dependem do tamanho do arquivo. Para arquivos pequenos e médios (até 10.000 chaves), o **Huffman Canônico** foi superior ao LZW tanto em taxa ($1,81\times/1,88\times$ vs $1,34\times/1,80\times$) quanto em tempo ($\approx 3\text{--}5\times$ mais rápido). Para arquivos grandes (100.000 chaves), o **LZW** superou o Huffman em taxa ($1,95\times$ vs $1,88\times$), pois seu dicionário de 65.536 entradas se torna totalmente preenchido e os padrões repetidos do arquivo binário da árvore são explorados eficientemente. O Huffman mantém uso de memória estável e tempo de execução mais baixo em todos os cenários.

Como trabalho futuro, seria interessante avaliar o impacto de formatos de arquivo alternativos (por exemplo, serialização em largura ao invés de pré-ordem) sobre a taxa de compressão, uma vez que nós do mesmo nível tendem a ter conteúdo similar e poderiam ser melhor comprimidos juntos.